11-system-design

Modul 11: System Design — Arsitektur Aplikasi untuk Pemula

Level: 🟢 Beginner → 🟡 Intermediate
Jam: 8 (4 minggu × 1 sesi)
Prasyarat: Bisa bikin REST API pakai Express.js atau framework lain
Output: System design diagram + dokumentasi arsitektur untuk proyek capstone

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Membedakan arsitektur monolitik vs microservices dan kapan pake masing-masing
- Merancang database yang ternormalisasi dengan indexing yang tepat
- Memahami caching strategies (Redis, CDN) dan kapan pakainya
- Menjelaskan CAP theorem dan milih trade-off yang tepat
- Merancang message queue sederhana untuk komunikasi antar service
- Membandingkan opsi hosting dan memahami CI/CD pipeline

Materi

Sesi	Topik
1	Arsitektur Aplikasi & Jaringan — monolitik vs microservice, DNS, HTTP/HTTPS, REST, load
2	Database Design — normalisasi (1NF/2NF/3NF), indexing (B-tree, composite), N+1 proble
3	Caching & CAP Theorem — caching strategies (cache aside, write through, write behind),
4	Message Queue & Hosting — RabbitMQ/Redis pub/sub, event-driven architecture, hosting

Output Akhir Modul

> **System Design Document** — diagram arsitektur capstone lengkap dengan penjelasan: pilihan arsitektur, desain database, strategi caching, dan rencana deployment.

AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- ”Explain this architecture diagram and suggest improvements”
- ”Find the bottleneck in this system design”
- ”Compare monolithic vs microservices for this use case”
- ”Generate a normalized database schema for this requirement”
- ”Which caching strategy fits this scenario and why?”
- ”Design a CI/CD pipeline for this tech stack”

Catatan

Modul ini nge-cover konsep system design yang langsung kepake di proyek capstone. Fokus ke praktik — lo gak perlu hafal teori akademik. Tiap sesi ada file sendiri dengan penjelasan detail, kode contoh, dan latihan.

Ditulis untuk siswa SMK RPL — fokus ke praktik, minimal teori akademik yang gak penting

Sesi 1: Arsitektur Aplikasi & Jaringan

> **Topik:** Monolitik vs Microservice, DNS, HTTP/HTTPS, REST, Load Balancer, Deployment Strategies

---

1. Monolitik vs Microservices

Monolitik

Satu kode buat semuanya — frontend, backend, database query, semuanya dalam satu repo, satu server.

```
graph TB
    subgraph Monolith["Aplikasi Monolitik"]
        A[Express Server<br/>/auth · /products · /orders · /api]
        B[Database Connector]
        C[(PostgreSQL)]
        A --> B --> C
    end
    D[Browser / Mobile] --> A
```

Ciri-ciri:

- Satu kode base, satu deployment
- Semua fitur dalam satu proses
- Scaling: tinggal clone server

Kelebihan: | Pro | Keterangan | |-----|------------| | □ Gampang di-start | Cocok buat proyek sekolah / MVP | | □ Testing sederhana | Cukup 1 integration test suite | | □ Deployment mudah | Jalanin 1 service aja | | □ Debugging langsung | Error di 1 tempat |

Kekurangan: | Cons | Keterangan | |------|------------| | □ 1 error matiin semua | Bug di fitur A bisa bikin fitur B ikut down | | □ Susah dipisah tim | 2 orang edit file sama → conflict | | □ Build & deploy lambat | Semakin gede kode, semakin lama |

Analogi: Mall dengan 1 tenant raksasa. Semua toko dalam satu ruangan — kalau toko makanan error, toko baju ikut tutup.

Microservices

Fitur dipisah jadi service-service kecil, masing-masing jalan sendiri.

```
graph LR
    subgraph Services["Microservices"]
        A[Auth Service<br/>Port 3001]
        B[Product Service<br/>Port 3002]
        C[Order Service<br/>Port 3003]
    end
    subgraph DBs["Databases"]
        D[(DB Auth)]
        E[(DB Product)]
        F[(DB Order)]
    end
    G[API Gateway<br/>Port 3000]
    H[Browser / Mobile App]

    A --> D
    B --> E
```

```
    C --> F
    H --> G --> A
    G --> B
    G --> C
```

Ciri-ciri:

- Tiap service punya database sendiri
- Komunikasi via HTTP/REST atau message broker
- Bisa pakai tech stack beda tiap service

Kelebihan: | Pro | Keterangan | |-----|------------| | □ Isolasi error | Service A mati, B & C tetap jalan | | □ Scaling per-service | Yang sibuk aja di-scale | | □ Tim independen | Tiap tim pegang service beda | | □ Build/deploy cepet | Kode kecil-kecil |

Kekurangan: | Cons | Keterangan | |------|------------| | □ Kompleksitas tinggi | Perlu API Gateway, service discovery, monitoring | | □ Debugging susah | Tracing request antar service | | □ Data consistency rumit | Transaksi lintas service butuh saga pattern |

Analogi: Mall dengan tenant terpisah. Toko baju punya pintu sendiri, kasir sendiri. Kalau toko makanan renovasi, lo tetap belanja baju.

Kapan Pilih Yang Mana?

```
graph TD
    Q1["Tim >5 orang?"] -->|Ya| Q2
    Q1 -->|Tidak| Mono1["Mulai dengan Monolitik"]
    Q2["Fitur >10?"] -->|Ya| Q3
    Q2 -->|Tidak| Mono1
    Q3["Ada fitur dengan traffic 10x lipat?"] -->|Ya| Micro["Pertimbangkan Micro
    Q3 -->|Tidak| Mono1
    Micro --> Warning["⚠ Microservices overkill untuk <50 user<br/>Mulai monolit
```

---

2. DNS Resolution

DNS (Domain Name System) = **buku telepon internet**. Ubah nama domain (`google.com`) jadi IP address (`142.250.64.78`).

```
sequenceDiagram
    participant User as Browser
    participant DNS as DNS Resolver
    participant Server as Web Server

    User->>DNS: "siapa IP-nya rpl-capstone.com?"
    DNS-->>User: "103.xxx.xxx.xxx"
```

```
    User->>Server: GET / HTTP/1.1
    Server-->>User: HTML response
```

Alur lengkap DNS lookup:

1. **Browser cache** — cek dulu di local cache browser
2. **OS cache** — cek di `/etc/hosts` atau cache OS
3. **DNS Resolver** — tanya ke ISP (biasanya pake Google DNS `8.8.8.8` atau Cloudflare `1.1.1.1`)
4. **Root DNS** → **TLD DNS** (.com, .org) → **Authoritative DNS** (provider hosting lo)
5. Balikin IP address

Kenapa lo peduli?

- DNS lookup butuh waktu 20-120ms — kena ke tiap request pertama
- Pake DNS caching biar cepet (TTL biasanya 300-86400 detik)
- Cloudflare DNS `1.1.1.1` lebih cepet dari ISP default

---

3. HTTP / HTTPS / TLS

HTTP (Hypertext Transfer Protocol)

Protokol buat komunikasi browser ↔ server. Format request-response.

```
GET /api/products HTTP/1.1
Host: rpl-capstone.com
User-Agent: Mozilla/5.0
Accept: application/json

---

HTTP/1.1 200 OK
Content-Type: application/json

[{ "id": 1, "name": "Buku A" }]
```

HTTPS = HTTP + TLS (SSL)

Data dikirim dalam bentuk **terenkripsi** — gak bisa dibaca orang di tengah jalan.

```
graph LR
    subgraph TanpaHTTPS["HTTP — Plain Text"]
        A1[Browser] -->|"GET /login<br/>password: rahasia123"| B1[Server]
        C1[Attacker] -.->|"BISA BACA □"| A1
```

```
    end
    subgraph DenganHTTPS["HTTPS — Encrypted"]
        A2[Browser] -->|"🔒 ENCRYPTED<br/>can't read"| B2[Server]
        C2[Attacker] -.->|"GABISA BACA 🔒"| A2
    end
```

Cara kerja TLS (simplified):

1. Client minta koneksi HTTPS
2. Server kirim **SSL Certificate** (berisi public key)
3. Client verifikasi certificate (apakah valid, gak expired)
4. Client & server bikin **session key** (symmetric encryption)
5. Semua data dienkripsi pake session key itu

TL;DR buat lo:

- HTTPS **WAJIB** buat production — apalagi kalau ada login / payment
- Pake **Let's Encrypt** (gratis) atau Cloudflare
- HTTP cuma buat local development

---

4. REST Principles

REST (Representational State Transfer) = gaya arsitektur API yang paling populer.

6 Prinsip REST (lo cukup paham 4 yang bold)

Prinsip	Maksud
Client-Server	Frontend & backend pisah
Stateless	Tiap request berdiri sendiri, gak perlu tahu request sebelumnya
Cacheable	Response bisa di-cache
Uniform Interface	Endpoint konsisten
Layered System	Client gak peduli di belakang ada apa aja
Code on Demand (optional)	Server kirim code executable

RESTful API Design

```
// 🔒 RESTful — endpoint ngikut resource
GET    /api/products          // list products
GET    /api/products/1        // detail product
POST   /api/products          // buat product baru
PUT    /api/products/1        // update product
DELETE /api/products/1        // hapus product
```

```
// ❌ BUKAN RESTful — endpoint pake verb
GET    /api/getProducts
POST   /api/createProduct
POST   /api/deleteProduct?id=1
```

HTTP Status Codes yang sering kepake:

Kode	Arti	Kapan
200	OK	Success GET, PUT, PATCH
201	Created	Success POST (data baru)
204	No Content	Success DELETE
400	Bad Request	Input gak valid
401	Unauthorized	Belum login
403	Forbidden	Gak punya akses
404	Not Found	Resource gak ada
429	Too Many Requests	Kena rate limit
500	Internal Server Error	Server error

---

5. Load Balancer

Load Balancer (LB) = alat yang ngedistribusikan traffic ke beberapa server.

```
graph TB
    Internet["🌐 Internet"] --> LB["⚖️ Load Balancer"]
    LB --> Web1["Web Server 1"]
    LB --> Web2["Web Server 2"]
    LB --> Web3["Web Server 3"]
    Web1 --> DB[(Database)]
    Web2 --> DB
    Web3 --> DB
```

Kenapa Perlu?

- 1 server bisa handle ~500-2000 request/detik
- Kalau user 10.000, 1 server jebol
- Dengan LB: traffic dibagi ke 3 server → masing-masing handle ~3.333 request

Algoritma Load Balancing

Metode	Cara Kerja	Analogi
Round Robin	Giliran: server 1 → 2 → 3 → 1	Kasir bank: nomor antria
Least Connections	Kirim ke server dengan koneksi paling sedikit	Pilih kasir antriannya pali
IP Hash	Client yang sama selalu ke server yang sama	Siswa ke ruang kelas yan

Untuk Capstone

Lo gak butuh load balancer beneran buat proyek sekolah. Tapi pahamin konsepnya:

- `pm2 start app.js -i max` → **cluster mode** — Node.js jalan pake semua CPU core
- Nanti di kerja beneran pake **NGINX** atau **AWS ALB**

```
graph LR
    subgraph TanpaLB["Tanpa LB"]
        A["Server<br/>1 core CPU"] --> B["□ Kewalahan"]
    end
    subgraph DenganPM2["Dengan PM2 Cluster"]
        C["Server<br/>4 core CPU"] --> D["Proses 1"]
        C --> E["Proses 2"]
        C --> F["Proses 3"]
        C --> G["Proses 4"]
        D --> H["□ Bagi-bagi request"]
        E --> H
        F --> H
        G --> H
    end
```

---

6. Deployment Strategies

Big Bang Deployment

```
graph LR
    A["v1.0 (Lama)"] --> B["□ Deploy Langsung"] --> C["v2.0 (Baru)"]
    D["User"] --> C
    style A fill:#ff9999
    style C fill:#99ff99
```

+ Paling sederhana
- Kalau error, semua user kena

Rolling Deployment

Update server satu per satu — gak semua kena downtime.

```
graph TB
    subgraph Tahap1["Tahap 1: 1 server di-update"]
        S1["□ v2.0"] --> LB["Load Balancer"]
        S2["□ v1.0"] --> LB
        S3["□ v1.0"] --> LB
    end
    subgraph Tahap2["Tahap 2: 2 server di-update"]
        S1_2["□ v2.0"] --> LB_2["Load Balancer"]
        S2_2["□ v2.0"] --> LB_2
        S3_2["□ v1.0"] --> LB_2
    end
    subgraph Selesai["Selesai: Semua v2.0"]
        S1_3["□ v2.0"] --> LB_3["Load Balancer"]
        S2_3["□ v2.0"] --> LB_3
        S3_3["□ v2.0"] --> LB_3
    end
```

Blue-Green Deployment

Dua lingkungan identik. Blue = live. Green = versi baru. Test dulu, baru switch.

```
graph LR
    LB["Load Balancer"] -->|"□ LIVE"| Blue["Blue Env (v1.0)"]
    LB -.->|"□ STANDBY"| Green["Green Env (v2.0)"]

    subgraph After["Setelah Switch"]
        LB2["Load Balancer"] -->|"□ LIVE"| Green2["Green Env (v2.0)"]
        LB2 -.->|"□ STANDBY"| Blue2["Blue Env (v1.0)"]
    end
```

+ Rollback instan — tinggal switch balik
- Butuh 2x resource (2 lingkungan)

Canary Deployment

Kirim versi baru ke **sebagian kecil user** dulu. Kalau aman, rollout ke semua.

```
graph TB
    LB["Load Balancer"] -->|"90% user"| Stable["Stable (v1.0)"]
    LB -->|"10% user"| Canary["Canary (v2.0)"]
    Canary --> Monitor["□ Monitor error rate"]
```

```
    Monitor -->|"□ Aman"| Full["Rollout ke 100%"]
    Monitor -->|"□ Error tinggi"| Rollback["Rollback ke v1.0"]
```

Latihan

1. **Diagram arsitektur:** Gambar arsitektur monolitik untuk aplikasi capstone lo (minimal: server, database, client). Terus gambar versi microservices-nya. Apa aja yang berubah?
2. **DNS & HTTPS:** Cek domain (atau IP) aplikasi capstone lo pake `nslookup` atau `dig`. Terus cek apakah udah pake HTTPS. Kalau belum, cari tau gimana cara pasang SSL pake Let's Encrypt.
3. **Desain REST API:** Dari fitur capstone lo, buat daftar endpoint REST yang lo butuhin. Tulis method (GET/POST/PUT/DELETE), path, dan status code response untuk tiap endpoint. Jangan lupa grouping berdasarkan resource.
4. **Strategi deployment:** Kalau lo deploy aplikasi capstone ke Vercel + Railway, termasuk strategi deployment apa (big bang, rolling, blue-green, canary)? Jelaskan kenapa.

Sesi 2: Database Design

> **Topik:** Normalisasi (1NF/2NF/3NF), Indexing (B-tree, Composite), N+1 Problem, Migration Strategies

---

1. Normalisasi Database

Normalisasi = proses **ngilangin data duplikat** dari database.

Kenapa Harus Dinormalisasi?

Tanpa normalisasi, tabel lo bakal kayak gini:

```sql
-- □ Tabel JELEK — data berulang
CREATE TABLE pesanan (
    id SERIAL PRIMARY KEY,
    produk VARCHAR(100),
    nama_pelanggan VARCHAR(100),
    alamat_pelanggan TEXT
);
```

```
-- Data:
-- 1 | Buku A  | Budi Santoso | Jl. Merdeka No.1
-- 2 | Pensil  | Budi Santoso | Jl. Merdeka No.1  ← duplikat!
-- 3 | Buku B  | Budi Santoso | Jl. Merdeka No.1  ← duplikat!
-- 4 | Buku A  | Siti Rahma   | Jl. Sudirman No.2
```

Masalah: Nama Budi Santoso + alamatnya ditulis 3 kali. Kalau Budi pindah rumah, lo harus ganti di 3 baris. Kalau lupa 1, data jadi inkonsisten.

1NF (First Normal Form)

Aturan: Setiap kolom isinya **1 nilai aja** (bukan array/list).

```
-- □ LANGKAH 1NF — kolom 'telepon' berisi banyak nomor
CREATE TABLE pelanggan_1nf (
    id SERIAL PRIMARY KEY,
    nama VARCHAR(100),
    telepon TEXT  -- "08123456789;087654321"
);

-- □ SETELAH 1NF — pisah ke tabel terpisah
CREATE TABLE pelanggan (
    id SERIAL PRIMARY KEY,
    nama VARCHAR(100)
);

CREATE TABLE telepon (
    id SERIAL PRIMARY KEY,
    pelanggan_id INT REFERENCES pelanggan(id),
    nomor VARCHAR(20)
);
```

2NF (Second Normal Form)

Aturan: Gak ada kolom yang tergantung sama **sebagian** primary key (berlaku kalau PK-nya composite key).

```
-- □ LANGKAH 2NF — PK composite (siswa_id, kelas_id)
-- Tapi 'nama_siswa' cuma tergantung sama siswa_id, bukan sama kelas_id
CREATE TABLE pendaftaran (
    siswa_id INT,
    kelas_id INT,
    nama_siswa VARCHAR(100),  -- □ tergantung siswa_id aja
    nama_kelas VARCHAR(50),   -- □ tergantung kelas_id aja
    tgl_daftar DATE,
    PRIMARY KEY (siswa_id, kelas_id)
```

```
);

-- □ SETELAH 2NF — pisah
CREATE TABLE siswa (
    id SERIAL PRIMARY KEY,
    nama VARCHAR(100)
);

CREATE TABLE kelas (
    id SERIAL PRIMARY KEY,
    nama VARCHAR(50)
);

CREATE TABLE pendaftaran (
    siswa_id INT REFERENCES siswa(id),
    kelas_id INT REFERENCES kelas(id),
    tgl_daftar DATE,
    PRIMARY KEY (siswa_id, kelas_id)
);
```

3NF (Third Normal Form)

Aturan: Kolom non-key gak boleh tergantung sama kolom non-key lain.

```
-- □ LANGKAH 3NF — 'kota' tergantung sama 'kode_pos', bukan sama 'id'
CREATE TABLE pelanggan_3nf (
    id SERIAL PRIMARY KEY,
    nama VARCHAR(100),
    kode_pos VARCHAR(5),
    kota VARCHAR(50)  -- □ tergantung kode_pos, bukan id
);

-- □ SETELAH 3NF — pisah
CREATE TABLE kode_pos (
    kode VARCHAR(5) PRIMARY KEY,
    kota VARCHAR(50)
);

CREATE TABLE pelanggan (
    id SERIAL PRIMARY KEY,
    nama VARCHAR(100),
    kode_pos VARCHAR(5) REFERENCES kode_pos(kode)
);
```

Ringkasan Visual

```
erDiagram
    Pelanggan ||--o{ Pesanan : punya
    Pelanggan {
        int id PK
        string nama
        string email
    }
    Pesanan ||--|| Pembayaran : dibayar
    Pesanan {
        int id PK
        int pelanggan_id FK
        date tgl_pesan
    }
    Pembayaran {
        int id PK
        int pesanan_id FK
        int jumlah
        string metode
    }
```

Rules of thumb (gak usah hafal nama bentuknya):

1. Ada kata diulang-ulang? → Pisahin ke tabel baru
2. Satu kolom berisi banyak nilai (pisah pake koma)? → Pisahin ke tabel baru
3. Foreign key > copy-paste data

---

2. Indexing

Index = fitur database buat nyari data **lebih cepet**, kayak indeks di buku.

Tanpa Index (Full Table Scan)

```
graph TD
    subgraph FullScan["Full Table Scan — 1 Juta Baris"]
        A["SELECT * FROM users WHERE email = 'budi@email.com'"]
        A --> B["Baris 1: bukan"]
        B --> C["Baris 2: bukan"]
        C --> D["..."]
        D --> E["Baris 873.402: bukan"]
        E --> F["Baris 873.403: budi@email.com □"]
```

```
    end
    style F fill:#99ff99
```

Database baca **semua baris** dari awal sampe akhir. Kalau 1 juta baris, bisa 500ms-5 detik.

Dengan Index (B-Tree)

```
graph TB
    subgraph BTree["B-Tree Index on email"]
        Root["Root<br/>'f'"] --> L1["'a' - 'e'"]
        Root --> L3["'f' - 'j'"]
        Root --> L4["'k' - 'z'"]
        L1 --> A["agus@mail.com → baris 1"]
        L3 --> B["budi@email.com → baris 873.403"]
        L4 --> C["citra@mail.com → baris 12.001"]
    end
    style B fill:#99ff99
```

Database pake struktur B-Tree — langsung lompat ke baris yang dicari. Butuh **<1ms**.

Kapan Pake Index?

```sql
-- ✅ Index kolom yang dipake WHERE
CREATE INDEX idx_users_email ON users (email);

-- ✅ Index foreign key (otomatis di PostgreSQL kalau pake REFERENCES)
CREATE INDEX idx_products_category ON products (category_id);

-- ✅ Index buat sorting
CREATE INDEX idx_orders_date ON orders (created_at DESC);

-- ❌ JANGAN index boolean / kolom dengan 2-3 nilai unik
-- ❌ JANGAN index tabel kecil (<1000 baris)
-- ❌ Hati-hati index di kolom yang sering UPDATE
```

Composite Index

Index di **lebih dari 1 kolom**. Urutan kolom **PENTING**.

```sql
-- Index composite: kolom dicari dari kiri ke kanan
CREATE INDEX idx_users_city_status ON users (city, status);

-- ✅ Berguna untuk query:
SELECT * FROM users WHERE city = 'Jakarta' AND status = 'active';
SELECT * FROM users WHERE city = 'Jakarta';
```

```sql
-- ❌ TIDAK berguna untuk:
SELECT * FROM users WHERE status = 'active';  -- kolom kiri (city) gak dipake
```

Analogi Index Composite: Buku telepon diurutkan (kota, nama). Kalau lo cari berdasarkan kota → cepet. Tapi kalau cari berdasarkan nama aja (tanpa kota) → lo tetap harus scan semua kota.

EXPLAIN — Cara Cek Performa Query

```sql
-- Cek apakah query pake index atau full scan
EXPLAIN ANALYZE SELECT * FROM users WHERE email = 'budi@email.com';
```

```
                                QUERY PLAN
-------------------------------------------------------------------------
 Index Scan using idx_users_email on users  (cost=...)
   Index Cond: ((email)::text = 'budi@email.com'::text)
   Execution Time: 0.123 ms
← Kalau liat "Index Scan" → bagus. Kalau "Seq Scan" → full table scan (lambat)
```

---

3. N+1 Query Problem

N+1 Problem = bug performa di ORM (Sequelize, Prisma, TypeORM) — bukan di SQL mentah.

Kejadiannya

```javascript
// ❌ INI SALAH — N+1 query!
// Lo punya 10 author, masing-masing punya banyak artikel

const authors = await Author.findAll(); // 1 query → dapet 10 author

for (const author of authors) {
  const articles = await Article.findAll({
    where: { authorId: author.id }
  });
  // 10 query lagi (1 per author)
  console.log(author.name, articles.length);
}
```

```
sequenceDiagram
    participant App as Express App
    participant DB as PostgreSQL

    App->>DB: SELECT * FROM authors
```

```
    DB-->>App: 10 authors
    loop Setiap author
        App->>DB: SELECT * FROM articles WHERE author_id = 1
        App->>DB: SELECT * FROM articles WHERE author_id = 2
        App->>DB: SELECT * FROM articles WHERE author_id = 3
        App->>DB: ... (10 query total)
    end
    Note right of DB: 11 query total 
```

Total query: 1 (ambil author) + 10 (ambil artikel per author) = **11 query**. Kalau author 1000 → 1001 query. Server lemess.

Solusi: Eager Loading (JOIN)

```
// ✅ INI BENER — JOIN dalam 1 query
const authors = await Author.findAll({
  include: [{ model: Article }]
});
// 1 query pake JOIN → selesai
```

```
sequenceDiagram
    participant App as Express App
    participant DB as PostgreSQL

    App->>DB: SELECT a.*, b.* FROM authors a LEFT JOIN articles b ON a.id = b.au
    DB-->>App: All data dalam 1 response
    Note right of App:  1 query doang!
```

```
-- Query yang di-generate ORM:
SELECT a.*, b.*
FROM authors a
LEFT JOIN articles b ON a.id = b.author_id;
```

Deteksi N+1

- Log query muncul query SAMA berulang-ulang
- API yang tadinya 50ms tiba-tiba jadi 2 detik pas data banyak
- Aktifin `logging: true` di Sequelize / `log: ['query']` di Prisma

Analogi: Lo mau fotokopi 10 halaman buku. Cara N+1: lo buka halaman 1 → ke mesin fotokopi → balik → buka halaman 2 → ... 10x bolak-balik. Cara JOIN: lo bawa bukunya ke mesin, fotokopi 10 halaman sekaligus.

---

4. Migration Strategies

Migration = cara **ngelola perubahan** struktur database secara terstruktur.

Kenapa Migration?

- Tim kerja bareng — perlu sinkron skema database
- Production udah jalan — gak bisa asal hapus kolom
- Version control buat database — bisa rollback kalau error

Contoh Migration (Knex.js)

```
// 20250101_create_users.js
exports.up = function (knex) {
  return knex.schema.createTable('users', (table) => {
    table.increments('id');
    table.string('email').unique().notNullable();
    table.string('password').notNullable();
    table.timestamps();
  });
};

exports.down = function (knex) {
  return knex.schema.dropTableIfExists('users');
};
```

```
// 20250102_add_phone_to_users.js
exports.up = function (knex) {
  return knex.schema.table('users', (table) => {
    table.string('phone', 15).nullable();
  });
};

exports.down = function (knex) {
  return knex.schema.table('users', (table) => {
    table.dropColumn('phone');
  });
};
```

Migration di Prisma

```
// schema.prisma
model User {
  id        Int      @id @default(autoincrement())
  email     String   @unique
```

```
  name      String?
  password  String
  phone     String?  // tambah kolom baru
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
```

```
# Jalanin migration
npx prisma migrate dev --name add_phone_field
```

Aturan Migration

Aturan	Penjelasan
1 migration = 1 perubahan	Jangan campur ”tambah kolom” + ”ganti tip
SELALU tulis `down`	Biar bisa rollback kalau error
Jangan edit migration yang udah di-commit	Buat migration BARU untuk perubahan berik
Test di staging dulu	Jangan langsung migrate di production
Backup database sebelum migrate	Safety net kalau ada yang salah

Migration Flow

```
graph TD
    A["Buat file migration<br/>20250101_create_users.js"] --> B["Jalankan: knex
    B --> C["Database berubah"]
    C --> D["Error? "]
    D -->|"Ya"| E["Jalankan: knex migrate:rollback"]
    D -->|"Tidak"| F[" Selesai"]
    E --> G["Perbaiki migration"]
    G --> B
```

Latihan

1. **Normalisasi skema:** Diberikan tabel berikut. Normalisasikan sampai 3NF. Tulis SQL CREATE TABLE untuk hasilnya.

```
-- Data pesanan rental mobil
-- 1 | Toyota Avanza | 500000 | Budi | budi@mail.com | 08123456789 | 2024-01-15
-- 2 | Honda Brio    | 400000 | Budi | budi@mail.com | 08123456789 | 2024-02-01 |
-- 3 | Toyota Avanza | 500000 | Siti | siti@mail.com | 087654321 | 2024-01-20 |
```

2. **Index optimization:** Dari aplikasi capstone lo, sebutkan 3 kolom yang paling cocok di-index. Tulis query CREATE INDEX-nya. Jelaskan kenapa milih kolom itu. Terus sebutkan 1 kolom yang **JANGAN** di-index dan kenapa.

3. **Fix N+1:** Kode di bawah punya N+1 problem. Tulis ulang pakai eager loading (JOIN) biar jadi 1 query.

```
const orders = await Order.findAll({ where: { userId: 1 } });

for (const order of orders) {
  const items = await OrderItem.findAll({
    where: { orderId: order.id }
  });
  console.log(`Order ${order.id}: ${items.length} items`);
}
```

4. **Migration strategy:** Lo punya tabel `users` yang mau ditambah kolom `refresh_token`. Tapi production udah jalan dengan 1000 user. Tulis migration file-nya (pakai knex atau Prisma) dengan up dan down. Apa yang harus lo perhatiin sebelum jalanin migration ini di production?

Sesi 3: Caching & CAP Theorem

> **Topik:** Caching Strategies (Cache Aside, Write Through, Write Behind), Redis Data Types, CDN, CAP Theorem, Consistency Models

---

1. Caching Strategies

Cache = tempat nyimpen data **sementara** biar akses berikutnya lebih cepet.

Database itu keras — baca dari disk, proses query, kirim data. Butuh waktu 10-200ms. Cache nyimpen hasil di **RAM** — akses <1ms.

```
graph LR
    A["Request"] --> B["Cache<br/>&lt;1ms"]
    B -->|"HIT "| C["Return data"]
    B -->|"MISS "| D["Database<br/>10-200ms"]
    D --> E["Simpan ke cache"]
    E --> C
```

Cache Aside (Paling Umum)

Aplikasi tanggung jawab penuh ngelola cache.

```
// CACHE ASIDE — aplikasi handle sendiri
async function getProduct(id) {
```

```
  // 1. Cek cache dulu
  const cached = await redis.get(`product:${id}`);

  if (cached) {
    console.log('CACHE HIT ');
    return JSON.parse(cached);
  }

  // 2. Miss — ambil dari DB
  console.log('CACHE MISS ');
  const product = await Product.findByPk(id);

  // 3. Simpan ke cache (expire 60 detik)
  await redis.setex(`product:${id}`, 60, JSON.stringify(product));

  // 4. Return
  return product;
}

// Kalau data di-update — HAPUS cache biar di-refresh
async function updateProduct(id, data) {
  const product = await Product.update(data, { where: { id } });
  await redis.del(`product:${id}`); // hapus cache
  return product;
}
```

```
sequenceDiagram
    participant App as Express App
    participant Cache as Redis
    participant DB as PostgreSQL

    App->>Cache: GET product:1
    Cache-->>App: MISS (null)
    App->>DB: SELECT * FROM products WHERE id=1
    DB-->>App: {id:1, name:"Buku A"}
    App->>Cache: SETEX product:1 60 "{...}"
    App-->>Client: {id:1, name:"Buku A"}

    Note over App,Cache: Request berikutnya...
    App->>Cache: GET product:1
    Cache-->>App: "{id:1, name:"Buku A"}"  HIT
```

Pro	Cons
Sederhana — mudah diimplementasi	Cache miss pertama tetap lambat
Cache cuma nyimpen data yang diminta	Perlu handle invalidation manual

Pro	Cons
✅ TTL bisa beda tiap key	❌ Data bisa basi (stale)

Write Through

Data ditulis ke **cache DAN database** secara bersamaan.

```
// WRITE THROUGH — tulis ke cache + DB barengan
async function createProduct(data) {
  // Tulis ke DB
  const product = await Product.create(data);

  // Langsung simpan ke cache juga
  await redis.set(`product:${product.id}`, JSON.stringify(product));

  return product;
}
```

```
sequenceDiagram
    participant App as Express App
    participant Cache as Redis
    participant DB as PostgreSQL

    App->>Cache: SET product:1 "{...}"
    App->>DB: INSERT INTO products ...
    Note over App: Data exist di cache & DB
    Cache-->>App: OK
    DB-->>App: OK
```

Pro	Cons
✅ Cache selalu up-to-date ✅ Read selalu HIT (asumsi data sering diakses)	❌ Write lebih lambat (2 operasi) ❌ Nyimpen data yang jarang diakses = boros n

Write Behind (Write Back)

Data ditulis ke **cache dulu**, trus di-sync ke database **nanti**.

```
// WRITE BEHIND — tulis ke cache dulu, DB belakangan
async function logUserAction(userId, action) {
  // Simpen di Redis dulu (cepat)
  await redis.lpush(`logs:${userId}`, JSON.stringify({ action, time: Date.now()

  // Nanti di-flush ke DB secara periodik
  // (background job tiap 5 menit)
```

```
}

// Background job — pindahin dari Redis ke DB
async function flushLogs() {
  const keys = await redis.keys('logs:*');
  for (const key of keys) {
    const logs = await redis.lrange(key, 0, -1);
    // Batch insert ke PostgreSQL
    await Log.bulkCreate(logs.map(JSON.parse));
    await redis.del(key);
  }
}
```

Pro	Cons
✅ Write sangat cepet (ke RAM) ✅ Bisa batch write ke DB	❌ Risiko data hilang kalau Redis crash sebelum di-flush ❌ Kompleksitas lebih tinggi

Kapan Pake Yang Mana?

Strategy	Pake Kalau
Cache Aside	✅ **Default — hampir semua kasus.** Data read-heavy, update jarang
Write Through	Data harus selalu konsisten antara cache & DB (misal: konfigurasi, session)
Write Behind	Log data, analytics, counter — data yang gak masalah kalau hilang sedikit

---

2. Redis Data Types

Redis bukan cuma SET/GET string. Banyak tipe data yang berguna.

Tipe	Perintah	Contoh Use Case
String	`SET, GET, SETEX`	Cache response API, session token
Hash	`HSET, HGET, HGETALL`	Profil user, object data
List	`LPUSH, RPOP, LRANGE`	Queue, log terbaru
Set	`SADD, SMEMBERS, SISMEMBER`	Tag, unique visitors
Sorted Set	`ZADD, ZRANGE, ZRANK`	Leaderboard, trending
Bitmap	`SETBIT, GETBIT`	Tracking user online hari ini

```
// STRING — cache product
await redis.setex('product:1', 60, JSON.stringify({ name: 'Buku A', price: 50000
```

```
// HASH — profil user (bisa akses field tertentu)
await redis.hset('user:1', { name: 'Budi', email: 'budi@mail.com', age: 17 });
const name = await redis.hget('user:1', 'name'); // "Budi"

// LIST — antrian notifikasi
await redis.lpush('notifications:user:1', 'Pesanan berhasil!');
await redis.lpush('notifications:user:1', 'Pembayaran diterima');
const recent = await redis.lrange('notifications:user:1', 0, 2); // 2 notif terb

// SORTED SET — leaderboard
await redis.zadd('leaderboard', 100, 'user:1'); // score 100
await redis.zadd('leaderboard', 85, 'user:2');
const top3 = await redis.zrevrange('leaderboard', 0, 2, 'WITHSCORES');
// [ 'user:1', '100', 'user:2', '85' ]
```

TTL (Time To Live)

```
// Set data dengan expire otomatis
await redis.setex('key', 3600, 'value'); // expire 1 jam
await redis.expire('key', 60); // set expire setelah set

// Cek sisa waktu
const ttl = await redis.ttl('key'); // detik tersisa
```

Best practice: SELALU kasih TTL. Cache tanpa TTL = bom memori.

---

3. CDN (Content Delivery Network)

CDN = jaringan server di seluruh dunia yang nyimpen **file statis** (gambar, CSS, JS, video) biar loading cepet.

```
graph TD
    subgraph TanpaCDN["Tanpa CDN"]
        U1["User Jepang"] -->|"Request gambar<br/>latency: 300ms"| S["Server di
        U2["User Indonesia"] -->|"Request gambar<br/>latency: 20ms"| S
    end

    subgraph DenganCDN["Dengan CDN (Cloudflare)"]
        U1_CDN["User Jepang"] -->|"Request"| E1["Edge Server Tokyo<br/>&lt;5ms"]
        U2_CDN["User Indonesia"] -->|"Request"| E2["Edge Server Jakarta<br/>&lt;
        E1 -->|"Cache MISS"| O["Origin Server"]
        E2 -->|"Cache HIT "| O
    end
```

Cloudflare

CDN gratis yang paling populer. Lo tinggal pointing DNS ke Cloudflare, otomatis:

- File statis di-cache di 300+ lokasi worldwide
- HTTPS otomatis
- DDoS protection
- Minification otomatis (CSS/JS di-kecilin)

Konfigurasi cache di Cloudflare:

```
Page Rules → Cache Level: Standard
- Cache static files: .jpg, .png, .css, .js, .woff2
- TTL otomatis (biasanya 30 hari)
- HTML biasanya BYPASS (gak di-cache) biar dinamis
```

Untuk capstone: Pasang Cloudflare di domain lo. Gratis + bikin app lo loading cepet + HTTPS.

---

4. CAP Theorem

CAP Theorem = teorema yang bilang: **sistem terdistribusi gak bisa punya ketiganya sekaligus.**

3 Sifat

Sifat	Maksud	Gampangnya
Consistency	Semua node liat data yang **sama**	Semua server jawa
Availability	Semua request **tetap dilayani**, walau ada node mati	Ada server mati? S
Partition Tolerance	Sistem tetep jalan walau koneksi antar node **putus**	Server A & B putus

Pilih 2 dari 3

```
graph TD
    C["C - Consistency<br/>Data sama di semua node"]
    A["A - Availability<br/>Tetap jawab walau ada mati"]
    P["P - Partition Tolerance<br/>Tetap jalan walau jaringan putus"]

    C --- Center["Pilih 2 dari 3"]
    A --- Center
    P --- Center

    CP["CP<br/>Bank, SQL Database"] -->|"Data HARUS akurat<br/>Kalau ragu, tolak
```

```
    AP["AP<br/>Sosial Media, DNS"] -->|"Lebih penting serve user<br/>Sinkron nan
    CA["CA<br/> TIDAK MUNGKIN"] -->|"Di real dunia,<br/>jaringan PASTI putus"|
```

Yang paling penting: Network partition (P) pasti terjadi — jaringan gak 100% reliable. Jadi pilihan lo cuma **CP** atau **AP**.

CP (Consistency + Partition Tolerance)

Sistem milih **data akurat** daripada **selalu tersedia**.

```
// CP Example — Bank transfer
async function transfer(senderId, receiverId, amount) {
  // Pake TRANSACTION — kalo gagal semua di-rollback
  const result = await db.transaction(async (trx) => {
    const sender = await User.findByPk(senderId, { transaction: trx });
    if (sender.balance < amount) throw new Error('Saldo kurang');

    await User.decrement({ balance: amount }, { where: { id: senderId }, transac
    await User.increment({ balance: amount }, { where: { id: receiverId }, trans
  });

  // Kalau ada error transaksi — saldo tetap konsisten
  return result;
}
```

Contoh CP: Sistem perbankan. Kalau koneksi ATM ke server pusat putus, ATM gak kasih uang daripada saldo gak akurat.

AP (Availability + Partition Tolerance)

Sistem milih **tetap jawab** daripada **data pasti akurat**.

```
// AP Example — Like postingan (eventual consistency)
async function likePost(userId, postId) {
  // Langsung return — gak perlu nunggu sinkron ke semua node
  await redis.incr(`likes:${postId}`);

  // Background job buat sync ke DB nanti
  await queue.add('syncLikes', { postId });

  return { likes: await redis.get(`likes:${postId}`) };
}
```

Contoh AP: Sosial media. Lo bisa liat postingan temen walau server lain belum update. Ntar juga sinkron (eventual consistency).

Kenapa Ini Penting Buat Lo?

Service	Pilih	Alasan
Top-up saldo	CP	Jangan sampe saldo ganda
Beli item	CP	Stok harus akurat
Feed artikel	AP	User gak peduli real-time
Komentar	AP	Lebih baik bisa komen walau delay
Login session	AP	User lebih penting bisa login

---

5. Consistency Models

Strong Consistency

Data selalu **up-to-date** di semua node. Setelah write, semua read langsung liat data baru.

```
User A: SET balance = 5000
                      → Semua node punya balance = 5000
User B: GET balance   → 5000 (pasti)
```

+ Gak ada data basi
- Lambat (perlu nunggu semua node sinkron)
- Availability turun kalau ada node mati

Eventual Consistency

Data boleh **beda sementara**, ntar juga sinkron.

```
User A: SET balance = 5000  (di Node 1)
User B: GET balance         → Masih 4000 (data basi, OK)
                             ↓ 5 detik kemudian...
User B: GET balance         → 5000 (udah sinkron)
```

+ Cepet, availability tinggi
+ Tetap serve request walau ada node mati
- Data bisa basi (stale read)

Kapan Pake?

Model	Contoh
Strong Consistency	Transaksi keuangan, booking tiket, stok barang
Eventual Consistency	Like count, view count, feed notifikasi, log

---

Latihan

1. **Caching strategy:** Aplikasi capstone lo punya endpoint `GET /api/products` yang dipanggil 1000×/detik. Data produk berubah setiap kali admin edit (mungkin 1×/jam). Strategi caching apa yang paling cocok? Tulis kode implementasinya pake Redis (cache aside atau write through). Jangan lupa TTL dan invalidation.
2. **Redis data type:** Lo perlu nyimpen **top 10 produk terlaris** yang update real-time setiap ada transaksi. Tipe data Redis apa yang cocok? Tulis kode buat nambahin score pas ada transaksi dan ngambil top 10.
3. **CAP analysis:** Dari aplikasi capstone lo, sebutkan 3 fitur dan tentukan apakah fitur itu harus CP atau AP. Jelaskan kenapa.

```
Contoh tabel:
| Fitur | Pilih (CP/AP) | Alasan |
|-------|---------------|--------|
| Login | AP        | User lebih penting bisa login daripada data session 100% sinkr
| ...   | ...           | ...    |
```

4. **CDN setup:** Domain capstone lo di-cloudflare.com. File statis (gambar produk, CSS, JS) udah di-cache. Tapi user complain gambar produk lama muncul padahal admin udah ganti gambar. Apa yang terjadi dan gimana solusinya?

Sesi 4: Message Queue & Hosting

Topik: Message Queue (RabbitMQ/Redis), Pub/Sub, Event-Driven Architecture, Hosting Comparison, CI/CD Pipeline

---

1. Message Queue

Message Queue = tempat antrian pesan antar service. Service A kirim pesan, Service B ambil dan proses.

```
graph LR
    A["Service A<br/>Producer"] -->|"Kirim pesan"| Q[("Message Queue<br/> Antri
    Q -->|"Ambil & proses"| B["Service B<br/>Consumer"]
    Q -->|"Ambil & proses"| C["Service C<br/>Consumer"]
```

Kenapa Perlu Queue?

Tanpa Queue	Dengan Queue
User klik ”daftar” → nunggu email verifikasi → loading 3 detik	User klik ”daftar” → langsung da
1000 user upload foto barengan → server lemess	1000 upload antri, diproses 1 pe
Service order harus nunggu service payment	Service order kirim pesan ”pesa

RabbitMQ — Contoh

```
// PRODUCER — kirim pesan ke queue
const amqp = require('amqplib');

async function sendEmailQueue(emailData) {
  const connection = await amqp.connect('amqp://localhost');
  const channel = await connection.createChannel();

  const queue = 'email_queue';
  await channel.assertQueue(queue, { durable: true });

  channel.sendToQueue(queue, Buffer.from(JSON.stringify(emailData)), {
    persistent: true // pesan gak ilang kalau server restart
  });

  console.log(`□ Pesan masuk antrian: ${emailData.to}`);
  await channel.close();
  await connection.close();
}

// CONSUMER — ambil & proses pesan dari queue
async function processEmailQueue() {
  const connection = await amqp.connect('amqp://localhost');
  const channel = await connection.createChannel();

  const queue = 'email_queue';
  await channel.assertQueue(queue, { durable: true });

  // Ambil 1 pesan setiap kali (jangan banjir)
  channel.prefetch(1);

  console.log('□ Menunggu pesan email...');
  channel.consume(queue, async (msg) => {
    const emailData = JSON.parse(msg.content.toString());

    try {
      // Kirim email (pake nodemailer atau API mail)
      await sendEmail(emailData);
```

```
      console.log(`📧 Email terkirim ke ${emailData.to}`);

      // Confirm ke RabbitMQ bahwa pesan udah diproses
      channel.ack(msg);
    } catch (error) {
      console.log(`❌ Gagal kirim email: ${error.message}`);
      // Kalau gagal, pesan balik ke queue atau kirim ke dead letter queue
      channel.nack(msg, false, true); // requeue
    }
  });
}
```

Redis sebagai Queue

Redis juga bisa dipake sebagai queue sederhana pake List.

```
// PRODUCER — push ke list
await redis.lpush('email_queue', JSON.stringify({ to: 'budi@mail.com', subject:

// CONSUMER — blok sampe ada pesan (BLPOP)
const result = await redis.brpop('email_queue', 0); // 0 = wait forever
const emailData = JSON.parse(result[1]);
```

RabbitMQ	Redis Queue
Feature-rich (dead letter, routing, exchange)	Sederhana
Message persist ke disk	Bisa ilang kalau gak di-set dengan bener
Cocok buat production skala besar	Cocok buat tugas/capstone
Butuh install RabbitMQ server	Udah include kalau lo pake Redis

---

2. Pub/Sub (Publish/Subscribe)

Pola komunikasi **1-to-many**: 1 publisher, banyak subscriber.

```
graph TB
    P["📤 Publisher<br/>Order Service"] -->|"event: order.created"| Broker["📨 Mes
    Broker -->|"subscribe"| S1["📧 Email Service<br/>Kirim invoice"]
    Broker -->|"subscribe"| S2["🔔 Notification Service<br/>Push notif ke user"]
    Broker -->|"subscribe"| S3["📊 Analytics Service<br/>Catat order baru"]
    Broker -->|"subscribe"| S4["📦 Inventory Service<br/>Kurangi stok"]
```

Contoh Redis Pub/Sub

```
// SUBSCRIBER — jalan di file terpisah
const subscriber = new Redis();

subscriber.subscribe('order:created', (err, count) => {
  console.log(`Subscribe ke ${count} channel`);
});

subscriber.on('message', (channel, message) => {
  const order = JSON.parse(message);
  console.log(`□ Order baru: #${order.id}`);

  if (channel === 'order:created') {
    // Kirim email
    sendEmail(order.userEmail, 'Pesanan berhasil dibuat!');
    // Update inventory
    updateStock(order.items);
  }
});

// PUBLISHER — pas user checkout
const publisher = new Redis();

app.post('/checkout', async (req, res) => {
  const order = await Order.create(req.body);

  // Publish event — semua subscriber bakal dapet
  await publisher.publish('order:created', JSON.stringify(order));

  res.json({ message: 'Pesanan berhasil!', orderId: order.id });
});
```

Event-Driven Architecture

Dengan pub/sub, aplikasi lo jadi **event-driven** — tiap service reaksi terhadap event, gak perlu saling tahu.

```
graph LR
    subgraph Events["Event Bus"]
        E1["user.registered"]
        E2["order.created"]
        E3["payment.received"]
        E4["email.sent"]
    end
```

```
    A["Auth Service"] -->|"publish"| E1
    B["Order Service"] -->|"publish"| E2
    C["Payment Service"] -->|"publish"| E3

    E1 --> D["Email Service"]
    E2 --> D
    E2 --> E["Inventory Service"]
    E3 --> F["Notification Service"]
```

+ Decoupling — service gak perlu tahu satu sama lain
+ Scalable — tinggal tambah subscriber kalau perlu
+ Resilient — satu service mati, yang lain tetap jalan
- Eventual consistency — data mungkin delay
- Debugging susah — flow gak linear

---

3. Hosting Comparison

Buat capstone / proyek sekolah, lo bisa pilih beberapa opsi:

Platform	Cocok Buat	Harga
Vercel	Frontend (React/Next.js)	Gratis (Hobby)
Railway	Backend (Express/API) + DB	Gratis $5 credit/bulan, bayar se
Biznet Gio	Full app Indonesia	Mulai ~50rb/bulan
DOKS (Digital Ocean K8s)	Production skala besar	~$12-15/bulan
VPS (Digital Ocean Droplet)	Full control	~$6/bulan

Rekomendasi Buat Capstone

```
graph TD
    A["Arsitektur Capstone"] --> B["Frontend: React/Next.js"]
    A --> C["Backend: Express.js API"]
    A --> D["Database: PostgreSQL"]
    A --> E["Cache: Redis"]

    B --> F["Deploy ke Vercel □<br/>Gratis, auto HTTPS"]
    C --> G["Deploy ke Railway □<br/>Gratis $5 credit"]
    D --> G
    E --> G

    style F fill:#99ff99
    style G fill:#99ff99
```

Checklist Deployment

□ Frontend di Vercel — tinggal push ke GitHub, auto deploy
□ Backend di Railway — connect repo, set env variable
□ Database PostgreSQL di Railway — built-in
□ Redis di Railway — built-in
□ Domain kustom — Cloudflare (gratis) pointing ke Vercel + Railway
□ SSL — otomatis dari Vercel & Cloudflare

---

4. CI/CD Pipeline

CI (Continuous Integration) = tiap kali lo push kode, otomatis di-test.
CD (Continuous Deployment) = kalau test lolos, otomatis di-deploy ke production.

```
graph LR
    A["□□ Push ke GitHub"] --> B["□ CI: Jalankan Test"]
    B -->|"□ Gagal"| C["□ Kirim notif error"]
    B -->|"□ Lolos"| D["□ Build app"]
    D --> E["□ Deploy ke staging"]
    E --> F["□ Test staging"]
    F -->|"□ Lolos"| G["□ Deploy ke production"]
    F -->|"□ Gagal"| C
    G --> H["□ Monitor"]
```

Contoh GitHub Actions CI/CD

```yaml
# .github/workflows/deploy.yml
name: Deploy

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with:
          node-version: 18
      - run: npm ci
```

```yaml
      - run: npm test

  deploy:
    needs: test  # jalan kalau test sukses
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Deploy to Railway
        uses: bervProject/railway-deploy@v1
        with:
          railway_token: ${{ secrets.RAILWAY_TOKEN }}
```

Untuk Capstone (Minimal)

Buat proyek capstone, CI/CD minimal yang perlu lo setup:

1. **GitHub repo** — source of truth
2. **Vercel** — auto deploy dari GitHub (tinggal connect repo, frontend otomatis deploy tiap push ke main)
3. **Railway** — auto deploy backend (connect repo, set environment variables)
4. **Environment variables** — simpen di dashboard Vercel & Railway, gak perlu `.env` di repo

```
graph TB
    subgraph Local["Local Development"]
        A["git push origin main"]
    end
    subgraph GitHub["GitHub"]
        B["Repository"]
        C["GitHub Actions<br/>Jalankan test"]
    end
    subgraph Deploy["Auto Deploy"]
        D["Vercel □<br/>Frontend"]
        E["Railway □<br/>Backend + DB"]
    end

    A --> B --> C
    C -->|"□ Test lolos"| D
    C -->|"□ Test lolos"| E
```

---

5. Arsitektur Lengkap Capstone

Semua konsep yang udah dipelajari — digabung jadi satu:

```
graph TB
    subgraph Client["Client"]
        Browser["🌐 Browser"]
        Mobile["📱 Mobile App"]
    end

    subgraph Frontend["Frontend — Vercel"]
        React["React / Next.js"]
    end

    subgraph CDN["CDN — Cloudflare"]
        CF["Caching static files<br/>SSL / DDoS protection"]
    end

    subgraph Backend["Backend — Railway"]
        API["Express API<br/>REST endpoints"]
        Worker["Background Worker<br/>Queue consumer"]
    end

    subgraph Services["Services & Cache"]
        Redis["Redis<br/>Cache + Queue"]
        DB[(PostgreSQL<br/>Database)]
    end

    subgraph CI_CD["CI/CD — GitHub Actions"]
        Test["Test Suite"]
        Build["Build"]
        Deploy["Deploy"]
    end

    Browser -->|"DNS"| CF
    Mobile -->|"DNS"| CF
    CF --> React
    React -->|"API calls"| API
    API --> DB
    API --> Redis
    API --> Worker
    Worker --> DB
    CI_CD -->|"auto deploy"| Frontend
    CI_CD -->|"auto deploy"| Backend
```

Latihan

1. **Queue scenario:** Di capstone lo, ada fitur kirim email verifikasi, upload gambar, dan export laporan. Fitur mana yang perlu pake queue? Gambar diagram arsitektur-nya pake mermaid.
2. **Pub/Sub design:** Aplikasi lo punya event `order.shipped` (pesanan dikirim). Service apa aja yang perlu subscribe ke event ini? Tulis kode publisher dan subscriber pake Redis Pub/Sub.
3. **Hosting plan:** Tulis rencana hosting buat capstone lo: pilih platform buat frontend, backend, database. Hitung estimasi biaya (pakai yang gratis dulu). Kasih alasan kenapa milih platform itu.

Tabel:

Komponen	Platform	Alasan	Estimasi Biaya
Frontend	Vercel	Gratis, auto deploy	Rp 0
Backend	...	...	...
Database	...	...	...
Domain	...	...	...

4. **CI/CD flow:** Deskripsikan pipeline CI/CD buat capstone lo dalam 5 langkah. Mulai dari ”push ke GitHub” sampai ”app live di production”. Kalau test gagal, apa yang harus terjadi? Tulis juga file `.github/workflows/deploy.yml` sederhana.